ECORESIDUOS DEL LLANO · OPTIMIZADOR LOGÍSTICO

Diagramas de secuencia

Un diagrama por historia de usuario implementada. Participantes: el actor Analista y clases u objetos del código; mensajes: métodos con parámetros, tipos y tipo de retorno.

Pág.	Diagrama
2	Transversal · Inicio de sesión
3	HU-01 · Gestión de empresas agropecuarias
4	HU-01 · Gestión de plantas
5	HU-02 · Costos, distancias y coeficientes
6	HU-03 · Gestión de oferta y demanda
7	HU-04 (parcial) · Guardar y recuperar escenarios
8	HU-05 · Visualización de la red logística
9	HU-06 · Resolución con el modelo exacto
10	HU-06 · ModeloTransporte y Simplex de dos fases
11	HU-07 · Configuración y ejecución del algoritmo genético
12	HU-08 (parcial) · Visualización de resultados
13	HU-09 · Análisis de sensibilidad
14	HU-10 · Simulación de escenarios
15	HU-10 · Guardar la simulación como escenarios y compararlos
16	HU-11 · Comparación exacto vs. algoritmo genético
17	HU-11 · Exportar el reporte comparativo en PDF

Transversal · Inicio de sesión

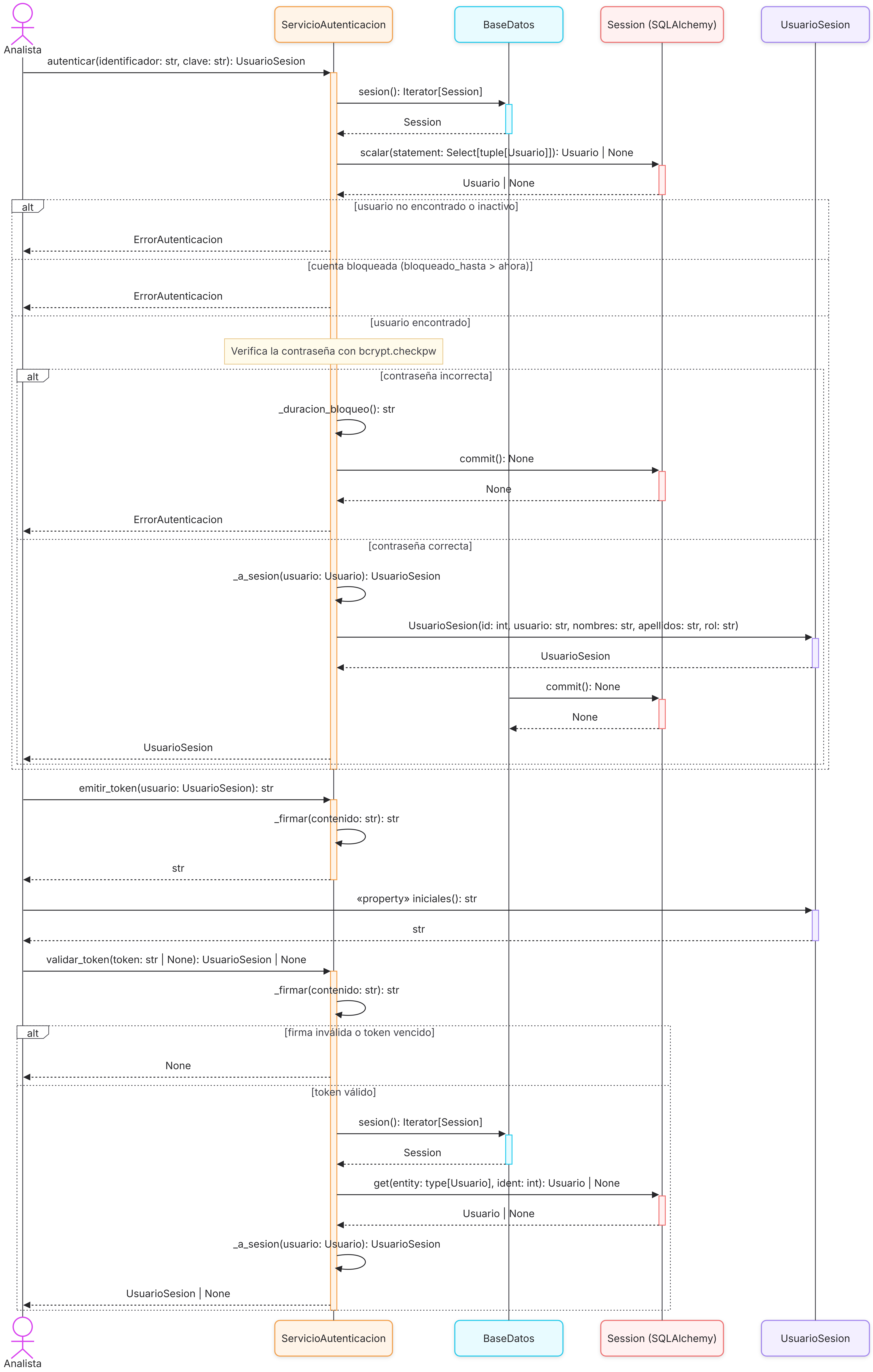

Analista
ServicioAutenticacion
BaseDatos
Session (SQLAlchemy)
UsuarioSesion
autenticar(identificador: str, clave: str): UsuarioSesion
sesion(): Iterator[Session]
Session
scalar(statement: Select[tuple[Usuario]]): Usuario | None
Usuario | None
alt
[usuario no encontrado o inactivo]
ErrorAutenticacion
[cuenta bloqueada (bloqueado_hasta > ahora)]
ErrorAutenticacion
[usuario encontrado]
Verifica la contraseña con bcrypt.checkpw
alt
[contraseña incorrecta]
_duracion_bloqueo(): str
commit(): None
None
ErrorAutenticacion
[contraseña correcta]
_a_sesion(usuario: Usuario): UsuarioSesion
UsuarioSesion(id: int, usuario: str, nombres: str, apellidos: str, rol: str)
UsuarioSesion
commit(): None
None
UsuarioSesion
emitir_token(usuario: UsuarioSesion): str
_firmar(contenido: str): str
str
«property» iniciales(): str
str
validar_token(token: str | None): UsuarioSesion | None
_firmar(contenido: str): str
alt
[firma inválida o token vencido]
None
[token válido]
sesion(): Iterator[Session]
Session
get(entity: type[Usuario], ident: int): Usuario | None
Usuario | None
_a_sesion(usuario: Usuario): UsuarioSesion
UsuarioSesion | None
Analista
ServicioAutenticacion
BaseDatos
Session (SQLAlchemy)
UsuarioSesion

DIAGRAMA DE SECUENCIA 2 DE 16

HU-01 · Gestión de empresas agropecuarias

Analista
ServicioEscenarios
ServicioRed
BaseDatos
Session (SQLAlchemy)
escenario: DatosEscenario
cargar(escenario_id: int): DatosEscenario
DatosEscenario
municipios(): list[ElementoCatalogo]
sesion(): Iterator[Session]
Session
scalars(statement: Select[tuple[Municipio]]): ScalarResult[Municipio]
ScalarResult[Municipio]
list[ElementoCatalogo]
tipos_residuo(): list[ElementoCatalogo]
sesion(): Iterator[Session]
Session
scalars(statement: Select[tuple[TipoResiduo]]): ScalarResult[TipoResiduo]
ScalarResult[TipoResiduo]
list[ElementoCatalogo]
«property» oferta(): np.ndarray
np.ndarray
alt
[crear empresa]
siguiente_codigo(prefijo: str): str
scalars(statement: Select[tuple[str]]): ScalarResult[str]
ScalarResult[str]
str
crear_empresa(escenario_id: int, datos: DatosEmpresa): None
_limpiar_empresa(datos: DatosEmpresa): DatosEmpresa
sesion(): Iterator[Session]
Session
_escenario(sesion: Session, escenario_id: int): Escenario
get(entity: type[Escenario], ident: int): Escenario | None
Escenario | None
_validar_empresa_unica(sesion: Session, datos: DatosEmpresa, excluir_id: int | None): None
scalar(statement: Select[tuple[Empresa]]): Empresa | None
Empresa | None
alt
[dato vacío o negativo, código o NIT repetido, escenario inexistente]
ErrorValidacion | ErrorNoEncontrado
[datos válidos]
loop
[por cada planta del escenario]
add(instance: Ruta): None
None
commit(): None
None
None
[editar empresa]
actualizar_empresa(empresa_id: int, id_vinculo: int, datos: DatosEmpresa): None
_limpiar_empresa(datos: DatosEmpresa): DatosEmpresa
sesion(): Iterator[Session]
Session
get(entity: type[Empresa], ident: int): Empresa | None
Empresa | None
get(entity: type[EscenarioEmpresa], ident: int): EscenarioEmpresa | None
EscenarioEmpresa | None
alt
[la empresa ya no existe]
ErrorNoEncontrado
[existe]
_validar_empresa_unica(sesion: Session, datos: DatosEmpresa, excluir_id: int | None): None
alt
[código o NIT repetido]
ErrorValidacion
[únicos]
commit(): None
None
None
[eliminar empresa]
eliminar_empresa(empresa_id: int): None
sesion(): Iterator[Session]
Session
get(entity: type[Empresa], ident: int): Empresa | None
Empresa | None
alt
[la empresa ya no existe]
ErrorNoEncontrado
[existe]
execute(statement: Delete): Result
Result
execute(statement: Delete): Result
Result
delete(instance: Empresa): None
None
commit(): None
None
None
Analista
ServicioEscenarios
ServicioRed
BaseDatos
Session (SQLAlchemy)
escenario: DatosEscenario

DIAGRAMA DE SECUENCIA 3 DE 16

HU-01 · Gestión de plantas

Analista
ServicioEscenarios
ServicioOptimizacion
ServicioRed
BaseDatos
Session (SQLAlchemy)
ejecucion: EjecucionExacta
cargar(escenario_id: int): DatosEscenario
DatosEscenario
ultima_exacta(escenario: DatosEscenario, ejecucion_id: int | None): EjecucionExacta | None
EjecucionExacta | None
municipios(): list[ElementoCatalogo]
list[ElementoCatalogo]
alt
[crear planta]
siguiente_codigo(prefijo: str): str
str
crear_planta(escenario_id: int, datos: DatosPlanta): None
_limpiar_planta(datos: DatosPlanta): DatosPlanta
sesion(): Iterator[Session]
Session
_escenario(sesion: Session, escenario_id: int): Escenario
get(entity: type[Escenario], ident: int): Escenario | None
Escenario | None
_validar_planta_unica(sesion: Session, codigo: str, excluir_id: int | None): None
scalar(statement: Select[tuple[Planta]]): Planta | None
Planta | None
alt
[sin código o municipio, valores negativos o código repetido]
ErrorValidacion
[datos válidos]
get(entity: type[Municipio], ident: int): Municipio | None
Municipio | None
loop
[por cada empresa del escenario]
add(instance: Ruta): None
None
commit(): None
None
None
[editar planta]
actualizar_planta(planta_id: int, id_vinculo: int, datos: DatosPlanta): None
_limpiar_planta(datos: DatosPlanta): DatosPlanta
sesion(): Iterator[Session]
Session
get(entity: type[Planta], ident: int): Planta | None
Planta | None
get(entity: type[EscenarioPlanta], ident: int): EscenarioPlanta | None
EscenarioPlanta | None
alt
[la planta ya no existe]
ErrorNoEncontrado
[existe]
_validar_planta_unica(sesion: Session, codigo: str, excluir_id: int | None): None
opt
[cambió el municipio]
get(entity: type[Municipio], ident: int): Municipio | None
Municipio | None
commit(): None
None
None
[eliminar planta]
eliminar_planta(planta_id: int): None
sesion(): Iterator[Session]
Session
get(entity: type[Planta], ident: int): Planta | None
Planta | None
alt
[la planta ya no existe]
ErrorNoEncontrado
[existe]
execute(statement: Delete): Result
Result
execute(statement: Delete): Result
Result
delete(instance: Planta): None
None
commit(): None
None
None
Analista
ServicioEscenarios
ServicioOptimizacion
ServicioRed
BaseDatos
Session (SQLAlchemy)
ejecucion: EjecucionExacta

HU-02 · Costos, distancias y coeficientes

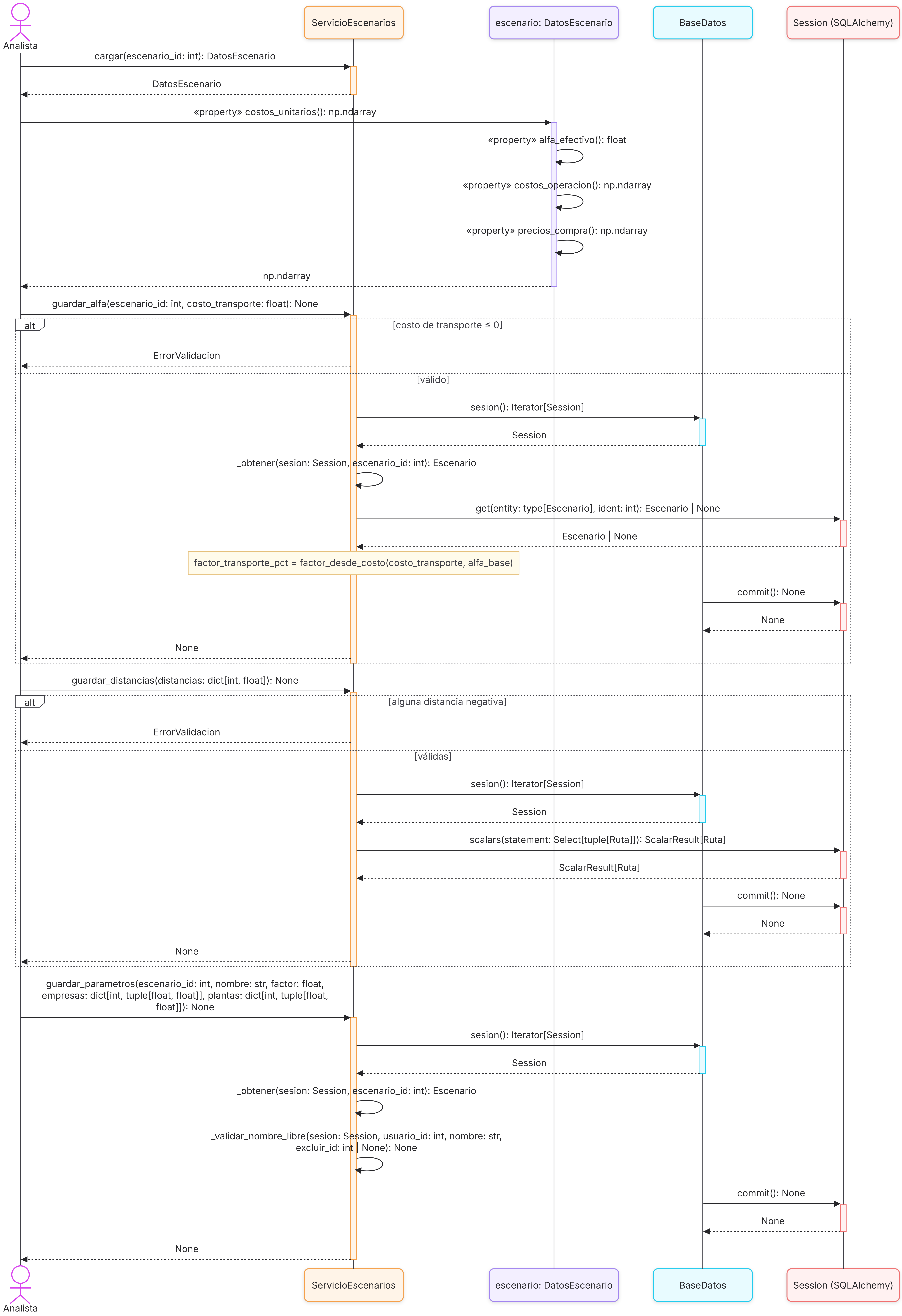

Analista
ServicioEscenarios
escenario: DatosEscenario
BaseDatos
Session (SQLAlchemy)
cargar(escenario_id: int): DatosEscenario
DatosEscenario
«property» costos_unitarios(): np.ndarray
«property» alfa_efectivo(): float
«property» costos_operacion(): np.ndarray
«property» precios_compra(): np.ndarray
np.ndarray
guardar_alfa(escenario_id: int, costo_transporte: float): None
alt
[costo de transporte ≤ 0]
ErrorValidacion
[válido]
sesion(): Iterator[Session]
Session
_obtener(sesion: Session, escenario_id: int): Escenario
get(entity: type[Escenario], ident: int): Escenario | None
Escenario | None
factor_transporte_pct = factor_desde_costo(costo_transporte, alfa_base)
commit(): None
None
None
guardar_distancias(distancias: dict[int, float]): None
alt
[alguna distancia negativa]
ErrorValidacion
[válidas]
sesion(): Iterator[Session]
Session
scalars(statement: Select[tuple[Ruta]]): ScalarResult[Ruta]
ScalarResult[Ruta]
commit(): None
None
None
guardar_parametros(escenario_id: int, nombre: str, factor: float, empresas: dict[int, tuple[float, float]], plantas: dict[int, tuple[float, float]]): None
sesion(): Iterator[Session]
Session
_obtener(sesion: Session, escenario_id: int): Escenario
_validar_nombre_libre(sesion: Session, usuario_id: int, nombre: str, excluir_id: int | None): None
commit(): None
None
None
Analista
ServicioEscenarios
escenario: DatosEscenario
BaseDatos
Session (SQLAlchemy)

DIAGRAMA DE SECUENCIA 5 DE 16

HU-03 · Gestión de oferta y demanda

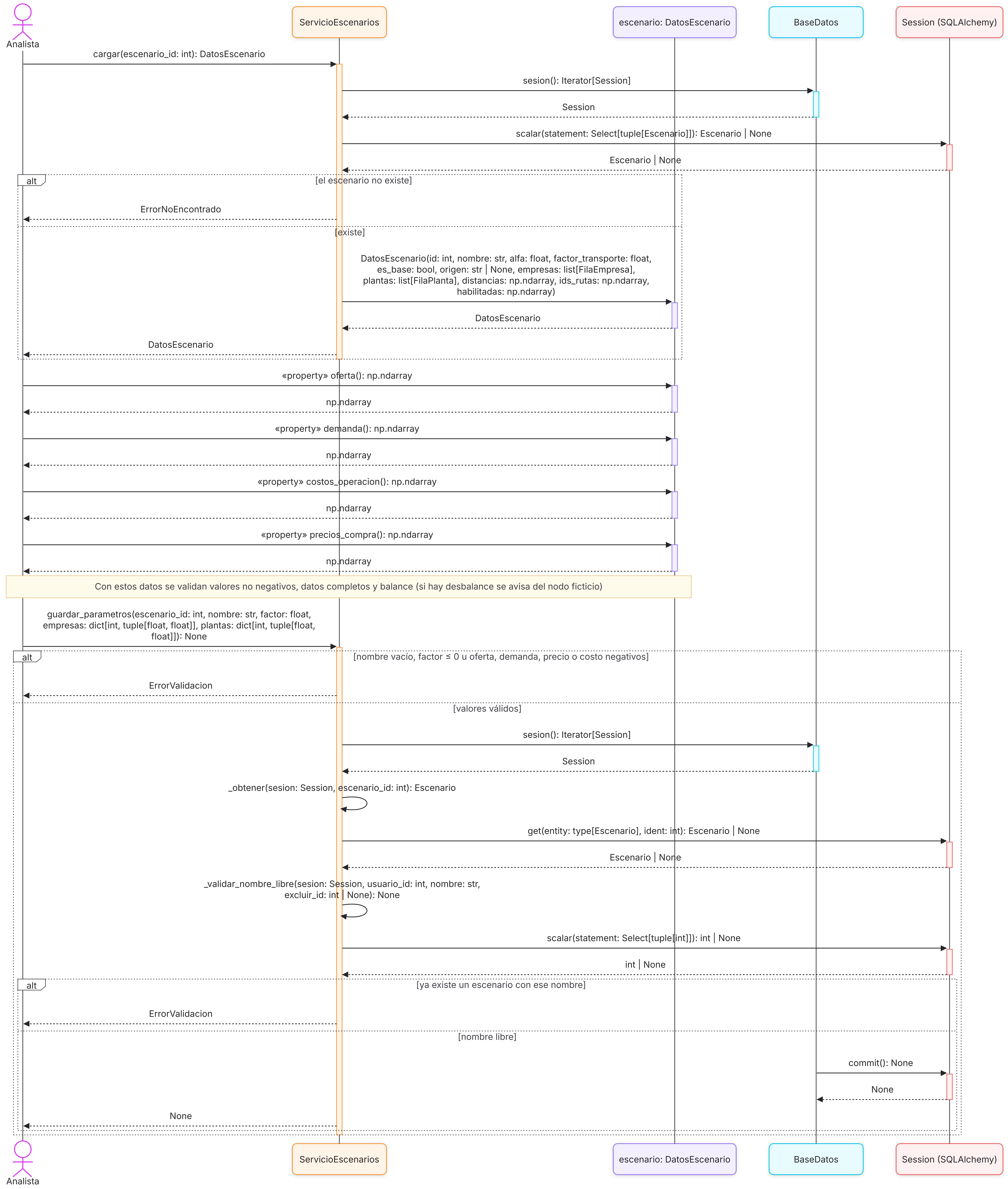

Analista
ServicioEscenarios
escenario: DatosEscenario
BaseDatos
Session (SQLAlchemy)
cargar(escenario_id: int): DatosEscenario
sesion(): Iterator[Session]
Session
scalar(statement: Select[tuple[Escenario]]): Escenario | None
Escenario | None
alt
[el escenario no existe]
ErrorNoEncontrado
[existe]
DatosEscenario(id: int, nombre: str, alfa: float, factor_transporte: float, es_base: bool, origen: str | None, empresas: list[FilaEmpresa], plantas: list[FilaPlanta], distancias: np.ndarray, ids_rutas: np.ndarray, habilitadas: np.ndarray)
DatosEscenario
DatosEscenario
«property» oferta(): np.ndarray
np.ndarray
«property» demanda(): np.ndarray
np.ndarray
«property» costos_operacion(): np.ndarray
np.ndarray
«property» precios_compra(): np.ndarray
np.ndarray
Con estos datos se validan valores no negativos, datos completos y balance (si hay desbalance se avisa del nodo ficticio)
guardar_parametros(escenario_id: int, nombre: str, factor: float, empresas: dict[int, tuple[float, float]], plantas: dict[int, tuple[float, float]]): None
alt
[nombre vacío, factor ≤ 0 u oferta, demanda, precio o costo negativos]
ErrorValidacion
[valores válidos]
sesion(): Iterator[Session]
Session
_obtener(sesion: Session, escenario_id: int): Escenario
get(entity: type[Escenario], ident: int): Escenario | None
Escenario | None
_validar_nombre_libre(sesion: Session, usuario_id: int, nombre: str, excluir_id: int | None): None
scalar(statement: Select[tuple[int]]): int | None
int | None
alt
[ya existe un escenario con ese nombre]
ErrorValidacion
[nombre libre]
commit(): None
None
None
Analista
ServicioEscenarios
escenario: DatosEscenario
BaseDatos
Session (SQLAlchemy)

DIAGRAMA DE SECUENCIA 6 DE 16

HU-04 (parcial) · Guardar y recuperar escenarios

Analista
ServicioEscenarios
BaseDatos
Session (SQLAlchemy)
escenario: DatosEscenario
listar(): list[ResumenEscenario]
sesion(): Iterator[Session]
Session
scalars(statement: Select[tuple[Escenario]]): ScalarResult[Escenario]
ScalarResult[Escenario]
list[ResumenEscenario]
resumenes(metodo: str): list[TarjetaEscenario]
sesion(): Iterator[Session]
Session
scalars(statement: Select[tuple[Escenario]]): ScalarResult[Escenario]
ScalarResult[Escenario]
execute(statement: Select[tuple[int, Decimal]]): Result
Result
list[TarjetaEscenario]
alt
[guardar un escenario nuevo o duplicar uno existente]
duplicar(escenario_id: int, usuario_id: int, nombre: str | None, descripcion: str | None): int
sesion(): Iterator[Session]
Session
scalar(statement: Select[tuple[Escenario]]): Escenario | None
Escenario | None
alt
[el escenario de origen no existe]
ErrorNoEncontrado
[existe]
opt
[sin nombre]
_nombre_copia(sesion: Session, usuario_id: int, base: str): str
_validar_nombre_libre(sesion: Session, usuario_id: int, nombre: str, excluir_id: int | None): None
alt
[nombre repetido]
ErrorValidacion
[nombre libre]
loop
[por cada ruta del escenario de origen]
add(instance: Ruta): None
None
add(instance: Escenario): None
None
flush(): None
None
commit(): None
None
int
[recuperar (abrir) un escenario]
cargar(escenario_id: int): DatosEscenario
sesion(): Iterator[Session]
Session
scalar(statement: Select[tuple[Escenario]]): Escenario | None
Escenario | None
alt
[el escenario no existe]
ErrorNoEncontrado
[existe]
DatosEscenario(id: int, nombre: str, alfa: float, factor_transporte: float, es_base: bool, origen: str | None, empresas: list[FilaEmpresa], plantas: list[FilaPlanta], distancias: np.ndarray, ids_rutas: np.ndarray, habilitadas: np.ndarray)
DatosEscenario
DatosEscenario
Analista
ServicioEscenarios
BaseDatos
Session (SQLAlchemy)
escenario: DatosEscenario

No implementado de HU-04: importar y exportar datos de la red en CSV, XLSX o JSON

DIAGRAMA DE SECUENCIA 7 DE 16

HU-05 · Visualización de la red logística

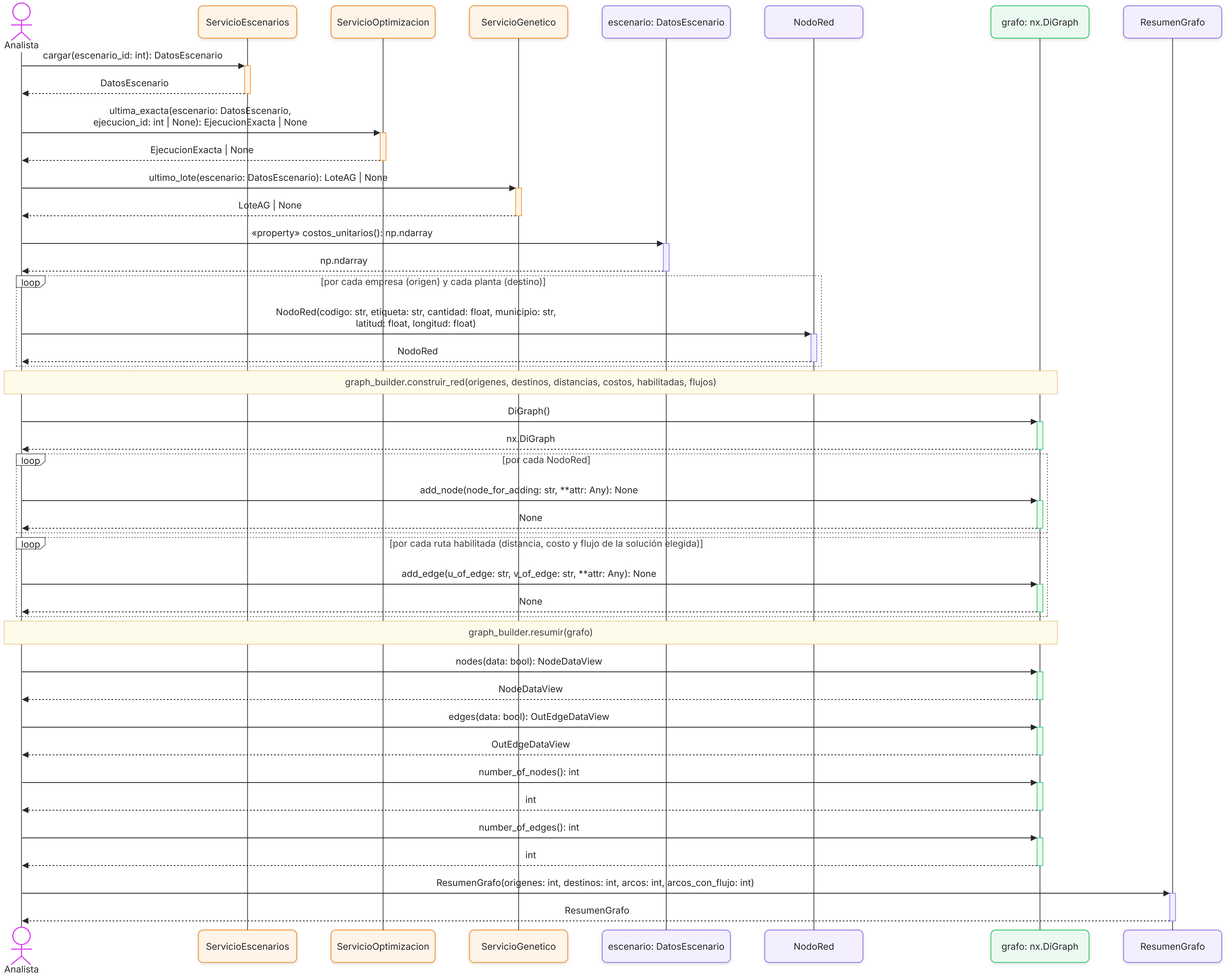

Analista
ServicioEscenarios
ServicioOptimizacion
ServicioGenetico
escenario: DatosEscenario
NodoRed
grafo: nx.DiGraph
ResumenGrafo
cargar(escenario_id: int): DatosEscenario
DatosEscenario
ultima_exacta(escenario: DatosEscenario, ejecucion_id: int | None): EjecucionExacta | None
EjecucionExacta | None
ultimo_lote(escenario: DatosEscenario): LoteAG | None
LoteAG | None
«property» costos_unitarios(): np.ndarray
np.ndarray
loop
[por cada empresa (origen) y cada planta (destino)]
NodoRed(codigo: str, etiqueta: str, cantidad: float, municipio: str, latitud: float, longitud: float)
NodoRed
graph_builder.construir_red(origenes, destinos, distancias, costos, habilitadas, flujos)
DiGraph()
nx.DiGraph
loop
[por cada NodoRed]
add_node(node_for_adding: str, **attr: Any): None
None
loop
[por cada ruta habilitada (distancia, costo y flujo de la solución elegida)]
add_edge(u_of_edge: str, v_of_edge: str, **attr: Any): None
None
graph_builder.resumir(grafo)
nodes(data: bool): NodeDataView
NodeDataView
edges(data: bool): OutEdgeDataView
OutEdgeDataView
number_of_nodes(): int
int
number_of_edges(): int
int
ResumenGrafo(origenes: int, destinos: int, arcos: int, arcos_con_flujo: int)
ResumenGrafo
Analista
ServicioEscenarios
ServicioOptimizacion
ServicioGenetico
escenario: DatosEscenario
NodoRed
grafo: nx.DiGraph
ResumenGrafo

DIAGRAMA DE SECUENCIA 8 DE 16

HU-06 · Resolución con el modelo exacto

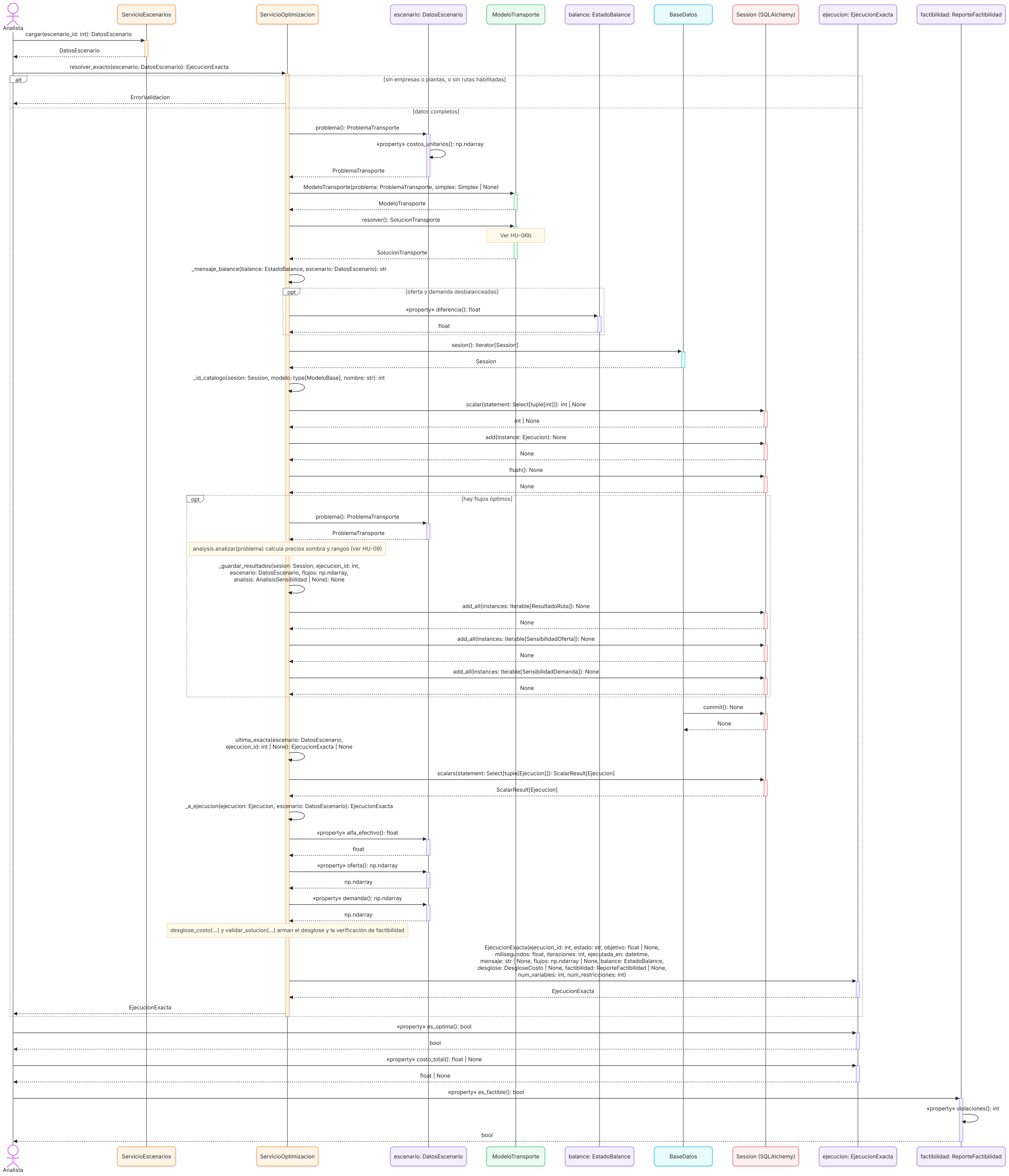
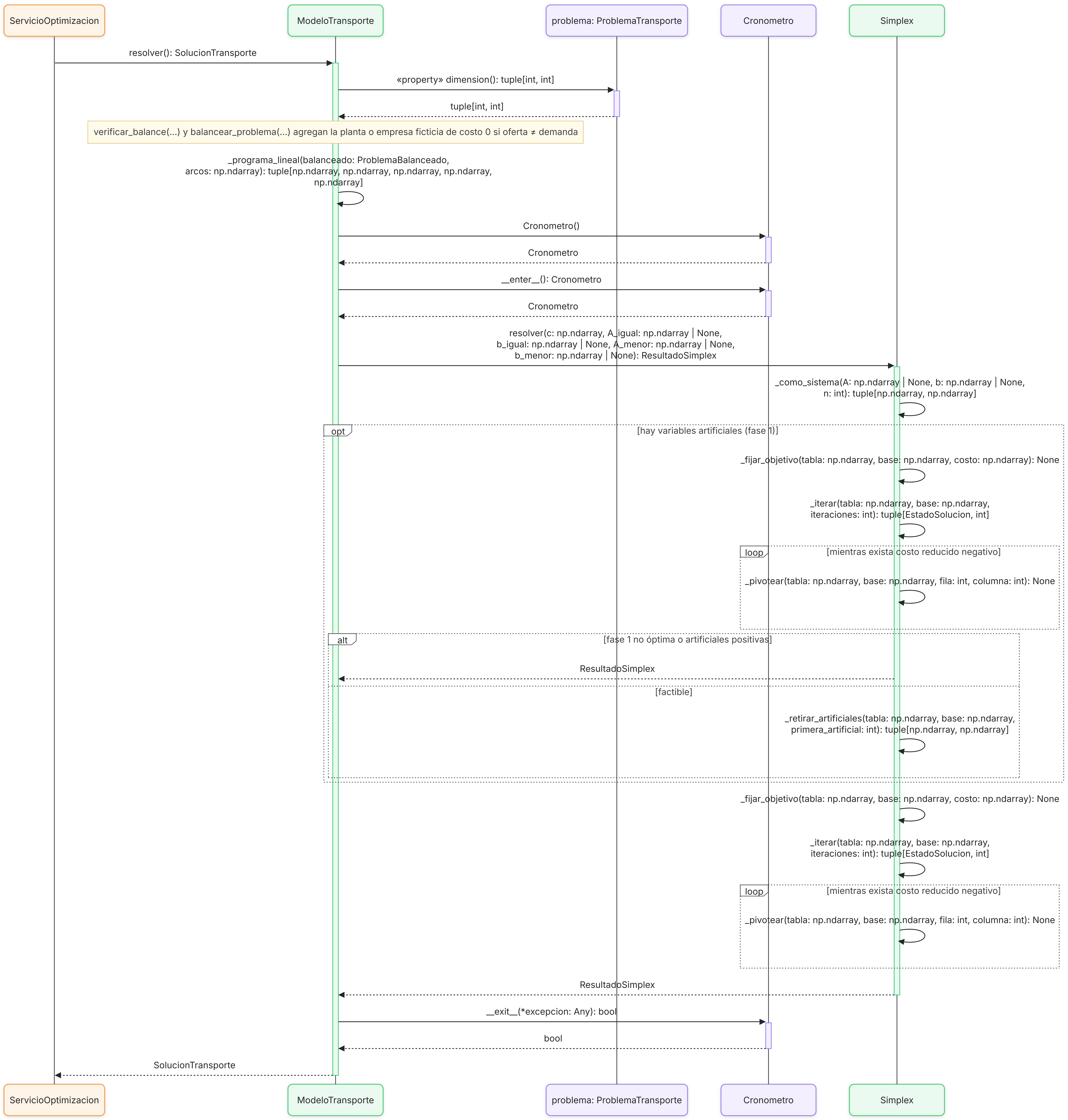

Analista
ServicioEscenarios
ServicioOptimizacion
escenario: DatosEscenario
ModeloTransporte
balance: EstadoBalance
BaseDatos
Session (SQLAlchemy)
ejecucion: EjecucionExacta
factibilidad: ReporteFactibilidad
cargar(escenario_id: int): DatosEscenario
DatosEscenario
resolver_exacto(escenario: DatosEscenario): EjecucionExacta
alt
[sin empresas o plantas, o sin rutas habilitadas]
ErrorValidacion
[datos completos]
problema(): ProblemaTransporte
«property» costos_unitarios(): np.ndarray
ProblemaTransporte
ModeloTransporte(problema: ProblemaTransporte, simplex: Simplex | None)
ModeloTransporte
resolver(): SolucionTransporte
Ver HU-06b
SolucionTransporte
_mensaje_balance(balance: EstadoBalance, escenario: DatosEscenario): str
opt
[oferta y demanda desbalanceadas]
«property» diferencia(): float
float
sesion(): Iterator[Session]
Session
_id_catalogo(sesion: Session, modelo: type[ModeloBase], nombre: str): int
scalar(statement: Select[tuple[int]]): int | None
int | None
add(instance: Ejecucion): None
None
flush(): None
None
opt
[hay flujos óptimos]
problema(): ProblemaTransporte
ProblemaTransporte
analysis.analizar(problema) calcula precios sombra y rangos (ver HU-09)
_guardar_resultados(sesion: Session, ejecucion_id: int, escenario: DatosEscenario, flujos: np.ndarray, analisis: AnalisisSensibilidad | None): None
add_all(instances: Iterable[ResultadoRuta]): None
None
add_all(instances: Iterable[SensibilidadOferta]): None
None
add_all(instances: Iterable[SensibilidadDemanda]): None
None
commit(): None
None
ultima_exacta(escenario: DatosEscenario, ejecucion_id: int | None): EjecucionExacta | None
scalars(statement: Select[tuple[Ejecucion]]): ScalarResult[Ejecucion]
ScalarResult[Ejecucion]
_a_ejecucion(ejecucion: Ejecucion, escenario: DatosEscenario): EjecucionExacta
«property» alfa_efectivo(): float
float
«property» oferta(): np.ndarray
np.ndarray
«property» demanda(): np.ndarray
np.ndarray
desglose_costo(...) y validar_solucion(...) arman el desglose y la verificación de factibilidad
EjecucionExacta(ejecucion_id: int, estado: str, objetivo: float | None, milisegundos: float, iteraciones: int, ejecutada_en: datetime, mensaje: str | None, flujos: np.ndarray | None, balance: EstadoBalance, desglose: DesgloseCosto | None, factibilidad: ReporteFactibilidad | None, num_variables: int, num_restricciones: int)
EjecucionExacta
EjecucionExacta
«property» es_optima(): bool
bool
«property» costo_total(): float | None
float | None
«property» es_factible(): bool
«property» violaciones(): int
bool
Analista
ServicioEscenarios
ServicioOptimizacion
escenario: DatosEscenario
ModeloTransporte
balance: EstadoBalance
BaseDatos
Session (SQLAlchemy)
ejecucion: EjecucionExacta
factibilidad: ReporteFactibilidad

DIAGRAMA DE SECUENCIA 9 DE 16

HU-06 · ModeloTransporte y Simplex de dos fases


ServicioOptimizacion
ModeloTransporte
problema: ProblemaTransporte
Cronometro
Simplex
resolver(): SolucionTransporte
«property» dimension(): tuple[int, int]
tuple[int, int]
verificar_balance(...) y balancear_problema(...) agregan la planta o empresa ficticia de costo 0 si oferta ≠ demanda
_programa_lineal(balanceado: ProblemaBalanceado, arcos: np.ndarray): tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray]
Cronometro()
Cronometro
__enter__(): Cronometro
Cronometro
resolver(c: np.ndarray, A_igual: np.ndarray | None, b_igual: np.ndarray | None, A_menor: np.ndarray | None, b_menor: np.ndarray | None): ResultadoSimplex
_como_sistema(A: np.ndarray | None, b: np.ndarray | None, n: int): tuple[np.ndarray, np.ndarray]
opt
[hay variables artificiales (fase 1)]
_fijar_objetivo(tabla: np.ndarray, base: np.ndarray, costo: np.ndarray): None
_iterar(tabla: np.ndarray, base: np.ndarray, iteraciones: int): tuple[EstadoSolucion, int]
loop
[mientras exista costo reducido negativo]
_pivotear(tabla: np.ndarray, base: np.ndarray, fila: int, columna: int): None
alt
[fase 1 no óptima o artificiales positivas]
ResultadoSimplex
[factible]
_retirar_artificiales(tabla: np.ndarray, base: np.ndarray, primera_artificial: int): tuple[np.ndarray, np.ndarray]
_fijar_objetivo(tabla: np.ndarray, base: np.ndarray, costo: np.ndarray): None
_iterar(tabla: np.ndarray, base: np.ndarray, iteraciones: int): tuple[EstadoSolucion, int]
loop
[mientras exista costo reducido negativo]
_pivotear(tabla: np.ndarray, base: np.ndarray, fila: int, columna: int): None
ResultadoSimplex
__exit__(*excepcion: Any): bool
bool
SolucionTransporte
ServicioOptimizacion
ModeloTransporte
problema: ProblemaTransporte
Cronometro
Simplex

HU-07 · Configuración y ejecución del algoritmo genético

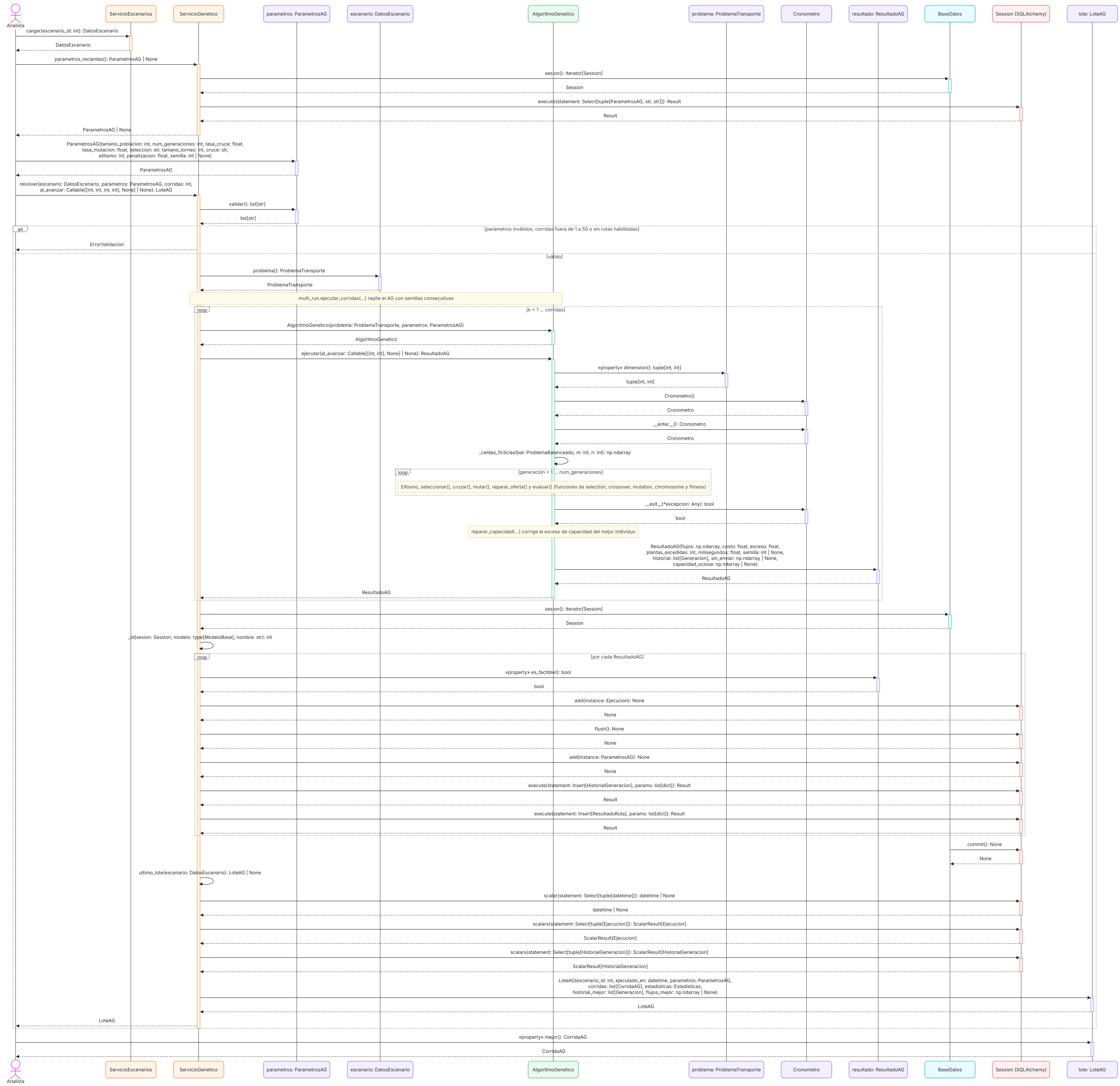

Analista
ServicioEscenarios
ServicioGenetico
parametros: ParametrosAG
escenario: DatosEscenario
AlgoritmoGenetico
problema: ProblemaTransporte
Cronometro
resultado: ResultadoAG
BaseDatos
Session (SQLAlchemy)
lote: LoteAG
cargar(escenario_id: int): DatosEscenario
DatosEscenario
parametros_recientes(): ParametrosAG | None
sesion(): Iterator[Session]
Session
execute(statement: Select[tuple[ParametrosAG, str, str]]): Result
Result
ParametrosAG | None
ParametrosAG(tamano_poblacion: int, num_generaciones: int, tasa_cruce: float, tasa_mutacion: float, seleccion: str, tamano_torneo: int, cruce: str, elitismo: int, penalizacion: float, semilla: int | None)
ParametrosAG
resolver(escenario: DatosEscenario, parametros: ParametrosAG, corridas: int, al_avanzar: Callable[[int, int, int, int], None] | None): LoteAG
validar(): list[str]
list[str]
alt
[parámetros inválidos, corridas fuera de 1 a 50 o sin rutas habilitadas]
ErrorValidacion
[válido]
problema(): ProblemaTransporte
ProblemaTransporte
multi_run.ejecutar_corridas(...) repite el AG con semillas consecutivas
loop
[k = 1 ... corridas]
AlgoritmoGenetico(problema: ProblemaTransporte, parametros: ParametrosAG)
AlgoritmoGenetico
ejecutar(al_avanzar: Callable[[int, int], None] | None): ResultadoAG
«property» dimension(): tuple[int, int]
tuple[int, int]
Cronometro()
Cronometro
__enter__(): Cronometro
Cronometro
_celdas_ficticias(bal: ProblemaBalanceado, m: int, n: int): np.ndarray
loop
[generación = 1 ... num_generaciones]
Elitismo, seleccionar(), cruzar(), mutar(), reparar_oferta() y evaluar() (funciones de selection, crossover, mutation, chromosome y fitness)
__exit__(*excepcion: Any): bool
bool
reparar_capacidad(...) corrige el exceso de capacidad del mejor individuo
ResultadoAG(flujos: np.ndarray, costo: float, exceso: float, plantas_excedidas: int, milisegundos: float, semilla: int | None, historial: list[Generacion], sin_enviar: np.ndarray | None, capacidad_ociosa: np.ndarray | None)
ResultadoAG
ResultadoAG
sesion(): Iterator[Session]
Session
_id(sesion: Session, modelo: type[ModeloBase], nombre: str): int
loop
[por cada ResultadoAG]
«property» es_factible(): bool
bool
add(instance: Ejecucion): None
None
flush(): None
None
add(instance: ParametrosAG): None
None
execute(statement: Insert[HistorialGeneracion], params: list[dict]): Result
Result
execute(statement: Insert[ResultadoRuta], params: list[dict]): Result
Result
commit(): None
None
ultimo_lote(escenario: DatosEscenario): LoteAG | None
scalar(statement: Select[tuple[datetime]]): datetime | None
datetime | None
scalars(statement: Select[tuple[Ejecucion]]): ScalarResult[Ejecucion]
ScalarResult[Ejecucion]
scalars(statement: Select[tuple[HistorialGeneracion]]): ScalarResult[HistorialGeneracion]
ScalarResult[HistorialGeneracion]
LoteAG(escenario_id: int, ejecutado_en: datetime, parametros: ParametrosAG, corridas: list[CorridaAG], estadisticas: Estadisticas, historial_mejor: list[Generacion], flujos_mejor: np.ndarray | None)
LoteAG
LoteAG
«property» mejor(): CorridaAG
CorridaAG
Analista
ServicioEscenarios
ServicioGenetico
parametros: ParametrosAG
escenario: DatosEscenario
AlgoritmoGenetico
problema: ProblemaTransporte
Cronometro
resultado: ResultadoAG
BaseDatos
Session (SQLAlchemy)
lote: LoteAG

DIAGRAMA DE SECUENCIA 11 DE 16

HU-08 (parcial) · Visualización de resultados

Analista
ServicioEscenarios
ServicioOptimizacion
ServicioGenetico
escenario: DatosEscenario
ejecucion: EjecucionExacta
factibilidad: ReporteFactibilidad
desglose: DesgloseCosto
balance: EstadoBalance
BaseDatos
Session (SQLAlchemy)
cargar(escenario_id: int): DatosEscenario
DatosEscenario
ultima_exacta(escenario: DatosEscenario, ejecucion_id: int | None): EjecucionExacta | None
EjecucionExacta | None
ultimo_lote(escenario: DatosEscenario): LoteAG | None
LoteAG | None
«property» es_optima(): bool
bool
«property» costo_total(): float | None
float | None
«property» oferta(): np.ndarray
np.ndarray
«property» demanda(): np.ndarray
np.ndarray
«property» diferencia(): float
float
ejecuciones_recientes(limite: int): list[ResumenEjecucion]
sesion(): Iterator[Session]
Session
scalars(statement: Select[tuple[Ejecucion]]): ScalarResult[Ejecucion]
ScalarResult[Ejecucion]
loop
[por cada ejecución (un lote del AG se muestra como su mejor corrida)]
_etiqueta(metodo: str, escenario: Escenario): str
list[ResumenEjecucion]
«property» costos_unitarios(): np.ndarray
np.ndarray
«property» es_factible(): bool
bool
«property» violaciones(): int
int
«property» total(): float
float
El PDF se exporta en HU-11. No implementado: exportar a CSV o XLSX

HU-09 · Análisis de sensibilidad

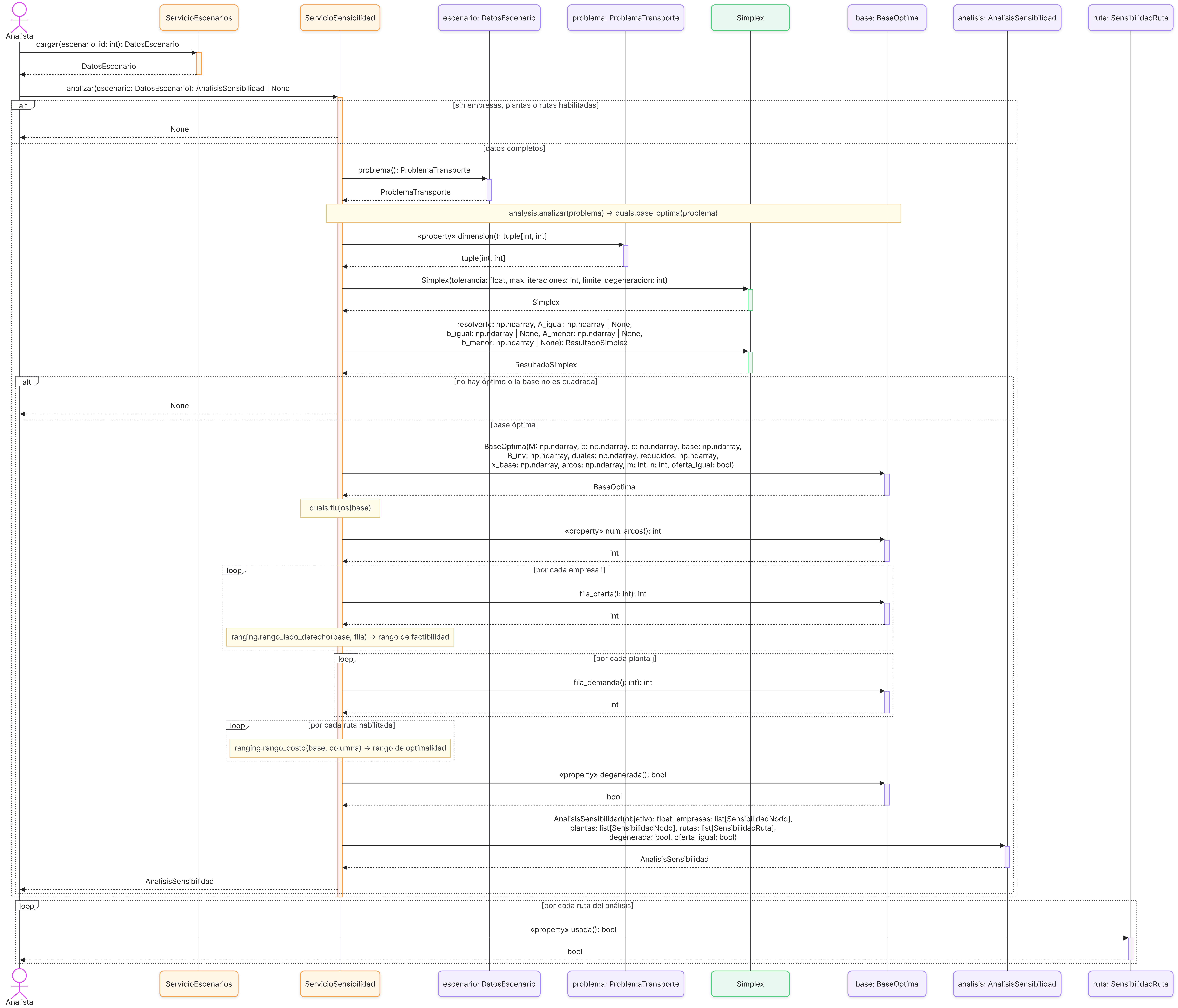

Analista
ServicioEscenarios
ServicioSensibilidad
escenario: DatosEscenario
problema: ProblemaTransporte
Simplex
base: BaseOptima
analisis: AnalisisSensibilidad
ruta: SensibilidadRuta
cargar(escenario_id: int): DatosEscenario
DatosEscenario
analizar(escenario: DatosEscenario): AnalisisSensibilidad | None
alt
[sin empresas, plantas o rutas habilitadas]
None
[datos completos]
problema(): ProblemaTransporte
ProblemaTransporte
analysis.analizar(problema) → duals.base_optima(problema)
«property» dimension(): tuple[int, int]
tuple[int, int]
Simplex(tolerancia: float, max_iteraciones: int, limite_degeneracion: int)
Simplex
resolver(c: np.ndarray, A_igual: np.ndarray | None, b_igual: np.ndarray | None, A_menor: np.ndarray | None, b_menor: np.ndarray | None): ResultadoSimplex
ResultadoSimplex
alt
[no hay óptimo o la base no es cuadrada]
None
[base óptima]
BaseOptima(M: np.ndarray, b: np.ndarray, c: np.ndarray, base: np.ndarray, B_inv: np.ndarray, duales: np.ndarray, reducidos: np.ndarray, x_base: np.ndarray, arcos: np.ndarray, m: int, n: int, oferta_igual: bool)
BaseOptima
duals.flujos(base)
«property» num_arcos(): int
int
loop
[por cada empresa i]
fila_oferta(i: int): int
int
ranging.rango_lado_derecho(base, fila) → rango de factibilidad
loop
[por cada planta j]
fila_demanda(j: int): int
int
loop
[por cada ruta habilitada]
ranging.rango_costo(base, columna) → rango de optimalidad
«property» degenerada(): bool
bool
AnalisisSensibilidad(objetivo: float, empresas: list[SensibilidadNodo], plantas: list[SensibilidadNodo], rutas: list[SensibilidadRuta], degenerada: bool, oferta_igual: bool)
AnalisisSensibilidad
AnalisisSensibilidad
loop
[por cada ruta del análisis]
«property» usada(): bool
bool
Analista
ServicioEscenarios
ServicioSensibilidad
escenario: DatosEscenario
problema: ProblemaTransporte
Simplex
base: BaseOptima
analisis: AnalisisSensibilidad
ruta: SensibilidadRuta

DIAGRAMA DE SECUENCIA 13 DE 16

HU-10 · Simulación de escenarios

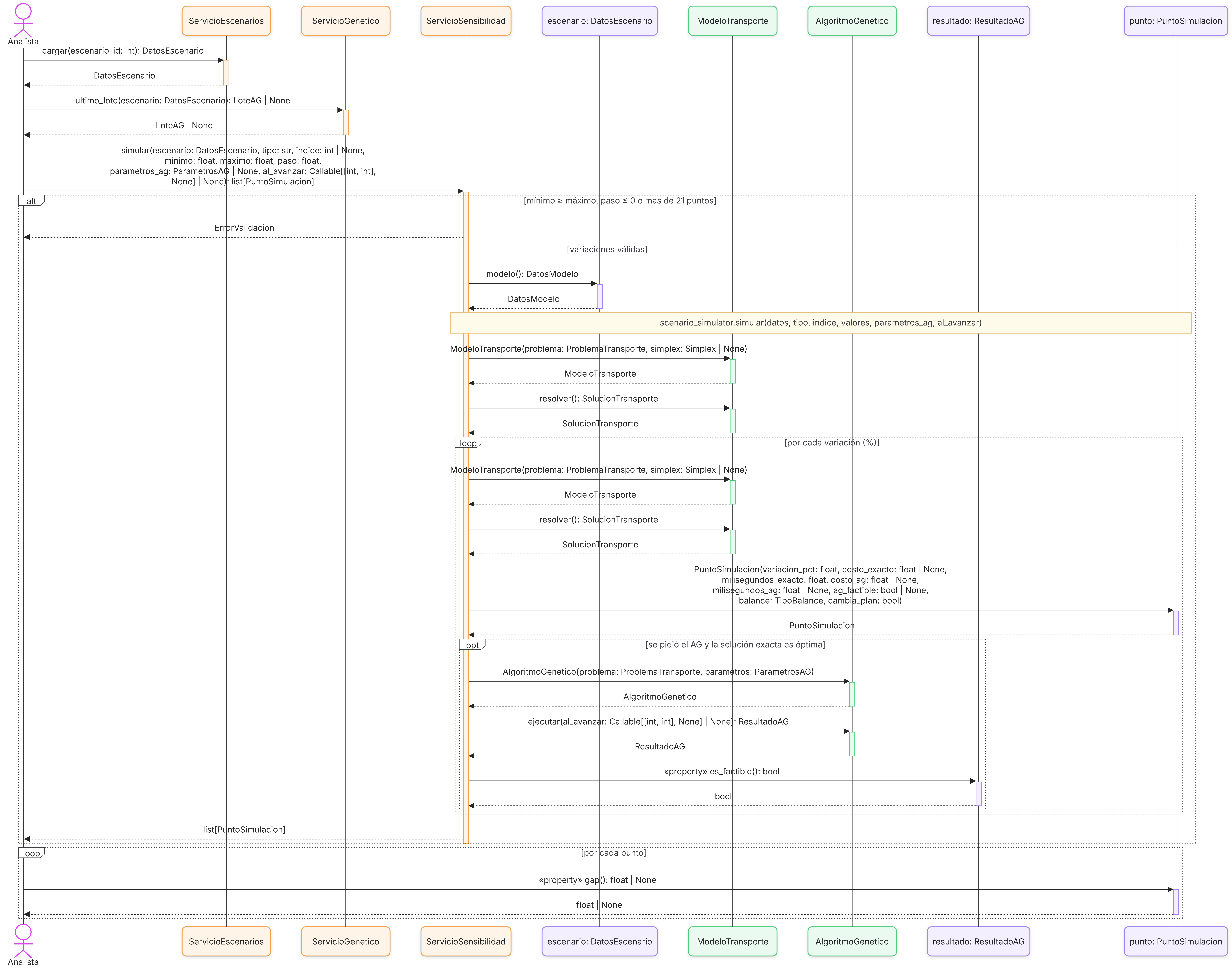
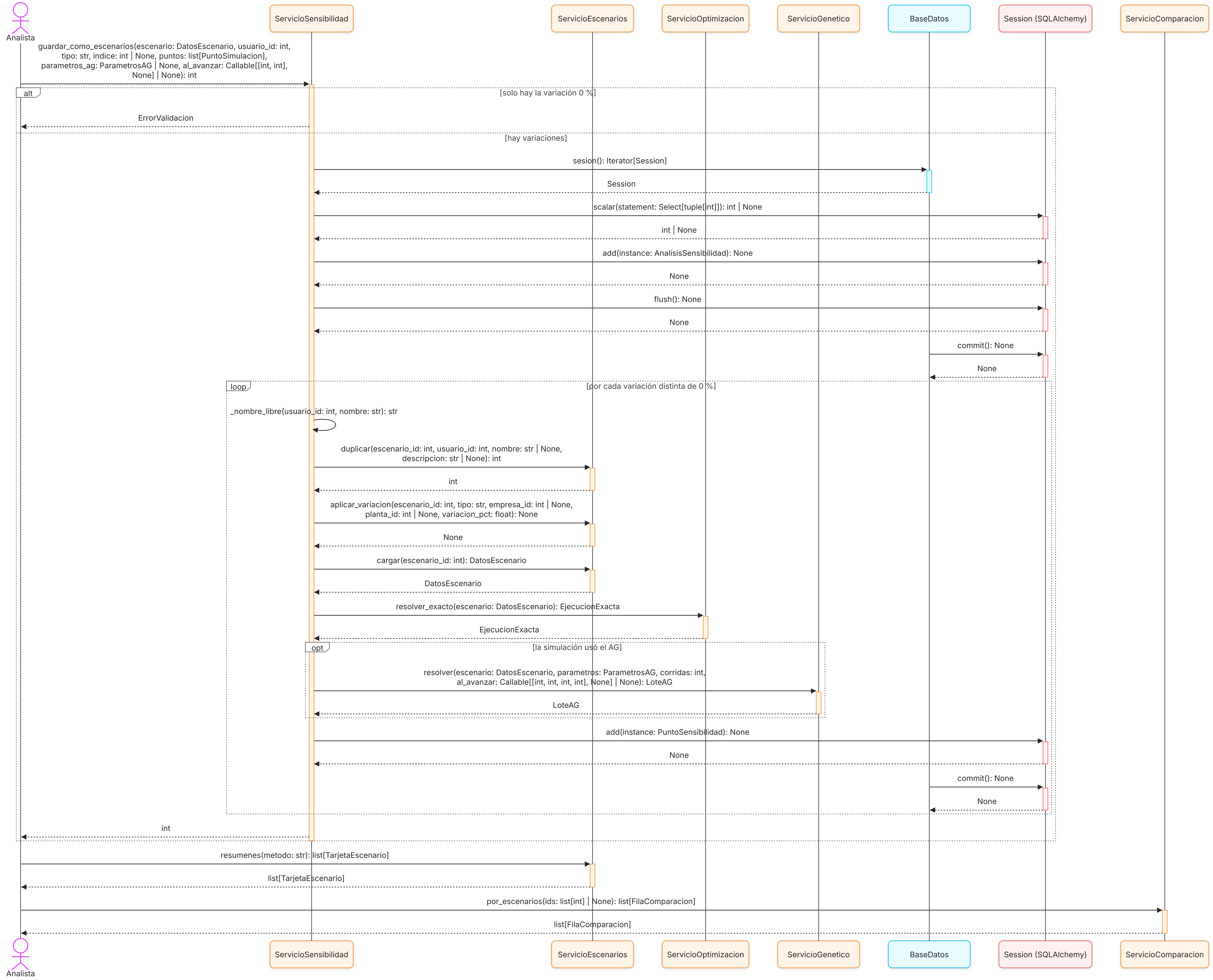

Analista
ServicioEscenarios
ServicioGenetico
ServicioSensibilidad
escenario: DatosEscenario
ModeloTransporte
AlgoritmoGenetico
resultado: ResultadoAG
punto: PuntoSimulacion
cargar(escenario_id: int): DatosEscenario
DatosEscenario
ultimo_lote(escenario: DatosEscenario): LoteAG | None
LoteAG | None
simular(escenario: DatosEscenario, tipo: str, indice: int | None, minimo: float, maximo: float, paso: float, parametros_ag: ParametrosAG | None, al_avanzar: Callable[[int, int], None] | None): list[PuntoSimulacion]
alt
[mínimo ≥ máximo, paso ≤ 0 o más de 21 puntos]
ErrorValidacion
[variaciones válidas]
modelo(): DatosModelo
DatosModelo
scenario_simulator.simular(datos, tipo, indice, valores, parametros_ag, al_avanzar)
ModeloTransporte(problema: ProblemaTransporte, simplex: Simplex | None)
ModeloTransporte
resolver(): SolucionTransporte
SolucionTransporte
loop
[por cada variación (%)]
ModeloTransporte(problema: ProblemaTransporte, simplex: Simplex | None)
ModeloTransporte
resolver(): SolucionTransporte
SolucionTransporte
PuntoSimulacion(variacion_pct: float, costo_exacto: float | None, milisegundos_exacto: float, costo_ag: float | None, milisegundos_ag: float | None, ag_factible: bool | None, balance: TipoBalance, cambia_plan: bool)
PuntoSimulacion
opt
[se pidió el AG y la solución exacta es óptima]
AlgoritmoGenetico(problema: ProblemaTransporte, parametros: ParametrosAG)
AlgoritmoGenetico
ejecutar(al_avanzar: Callable[[int, int], None] | None): ResultadoAG
ResultadoAG
«property» es_factible(): bool
bool
list[PuntoSimulacion]
loop
[por cada punto]
«property» gap(): float | None
float | None
Analista
ServicioEscenarios
ServicioGenetico
ServicioSensibilidad
escenario: DatosEscenario
ModeloTransporte
AlgoritmoGenetico
resultado: ResultadoAG
punto: PuntoSimulacion

DIAGRAMA DE SECUENCIA 14 DE 16

HU-10 · Guardar la simulación como escenarios y compararlos

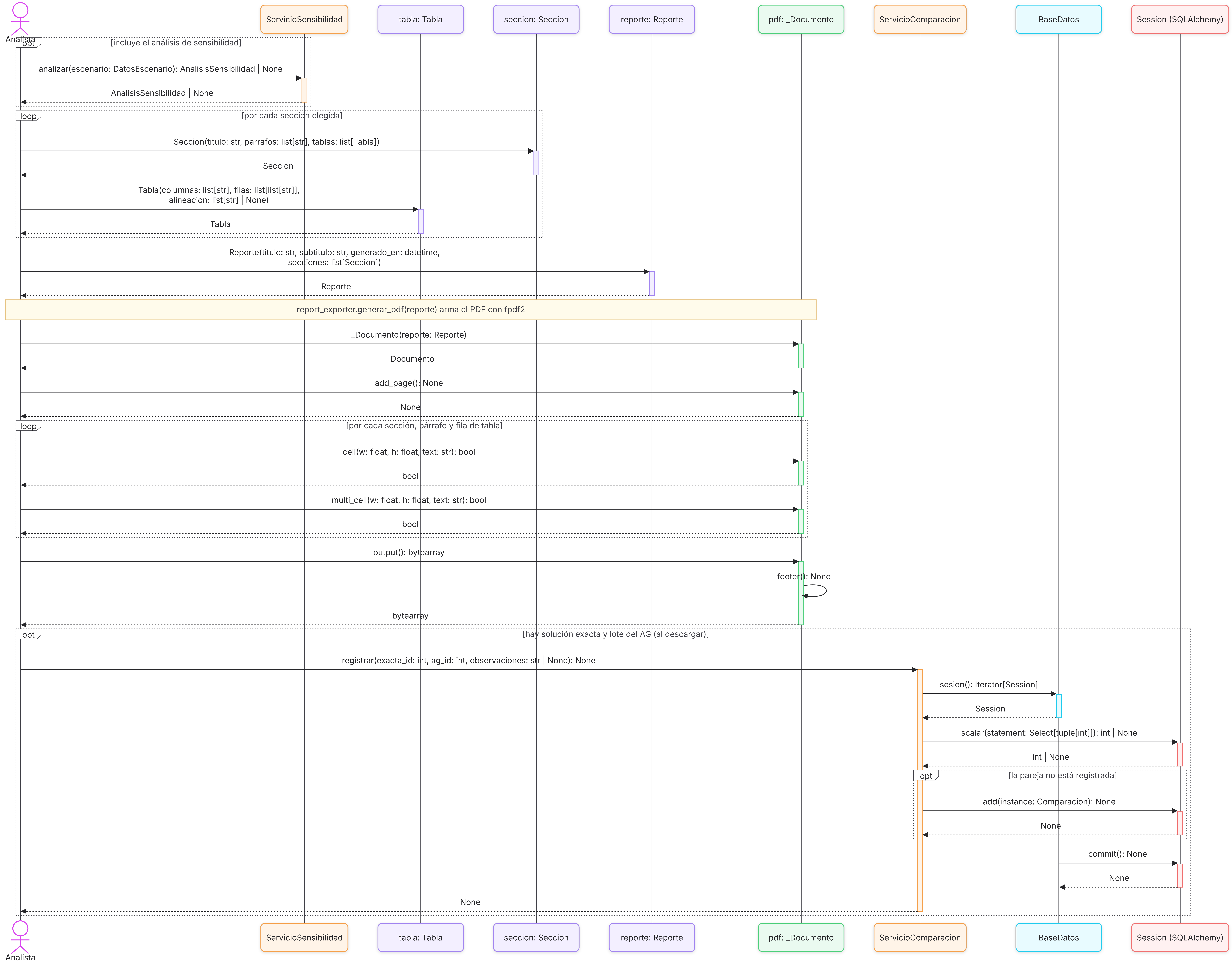

Analista
ServicioSensibilidad
ServicioEscenarios
ServicioOptimizacion
ServicioGenetico
BaseDatos
Session (SQLAlchemy)
ServicioComparacion
guardar_como_escenarios(escenario: DatosEscenario, usuario_id: int, tipo: str, indice: int | None, puntos: list[PuntoSimulacion], parametros_ag: ParametrosAG | None, al_avanzar: Callable[[int, int], None] | None): int
alt
[solo hay la variación 0 %]
ErrorValidacion
[hay variaciones]
sesion(): Iterator[Session]
Session
scalar(statement: Select[tuple[int]]): int | None
int | None
add(instance: AnalisisSensibilidad): None
None
flush(): None
None
commit(): None
None
loop
[por cada variación distinta de 0 %]
_nombre_libre(usuario_id: int, nombre: str): str
duplicar(escenario_id: int, usuario_id: int, nombre: str | None, descripcion: str | None): int
int
aplicar_variacion(escenario_id: int, tipo: str, empresa_id: int | None, planta_id: int | None, variacion_pct: float): None
None
cargar(escenario_id: int): DatosEscenario
DatosEscenario
resolver_exacto(escenario: DatosEscenario): EjecucionExacta
EjecucionExacta
opt
[la simulación usó el AG]
resolver(escenario: DatosEscenario, parametros: ParametrosAG, corridas: int, al_avanzar: Callable[[int, int, int, int], None] | None): LoteAG
LoteAG
add(instance: PuntoSensibilidad): None
None
commit(): None
None
int
resumenes(metodo: str): list[TarjetaEscenario]
list[TarjetaEscenario]
por_escenarios(ids: list[int] | None): list[FilaComparacion]
list[FilaComparacion]
Analista
ServicioSensibilidad
ServicioEscenarios
ServicioOptimizacion
ServicioGenetico
BaseDatos
Session (SQLAlchemy)
ServicioComparacion

HU-11 · Comparación exacto vs. algoritmo genético

Analista
ServicioEscenarios
ServicioOptimizacion
ServicioGenetico
ServicioComparacion
BaseDatos
Session (SQLAlchemy)
fila: FilaComparacion
lote: LoteAG
ModeloTransporte
AlgoritmoGenetico
resultado: ResultadoAG
cargar(escenario_id: int): DatosEscenario
DatosEscenario
ultima_exacta(escenario: DatosEscenario, ejecucion_id: int | None): EjecucionExacta | None
EjecucionExacta | None
ultimo_lote(escenario: DatosEscenario): LoteAG | None
LoteAG | None
«property» mejor(): CorridaAG
CorridaAG
por_escenarios(ids: list[int] | None): list[FilaComparacion]
sesion(): Iterator[Session]
Session
scalars(statement: Select[tuple[Escenario]]): ScalarResult[Escenario]
ScalarResult[Escenario]
scalars(statement: Select[tuple[Ejecucion]]): ScalarResult[Ejecucion]
ScalarResult[Ejecucion]
loop
[por cada escenario]
FilaComparacion(escenario_id: int, escenario: str, es_base: bool, exacta_id: int | None, costo_exacto: float | None, milisegundos_exacto: float | None, ag_id: int | None, costo_ag: float | None, milisegundos_ag: float | None, corridas_ag: int)
FilaComparacion
list[FilaComparacion]
loop
[por cada fila]
«property» gap(): float | None
float | None
opt
[medir escalabilidad (10 × 4, 50 × 10 y 100 × 20)]
metrics.medir_escalabilidad(tamanos, parametros, semilla) con instancias aleatorias
loop
[por cada tamaño]
ModeloTransporte(problema: ProblemaTransporte, simplex: Simplex | None)
ModeloTransporte
resolver(): SolucionTransporte
SolucionTransporte
AlgoritmoGenetico(problema: ProblemaTransporte, parametros: ParametrosAG)
AlgoritmoGenetico
ejecutar(al_avanzar: Callable[[int, int], None] | None): ResultadoAG
ResultadoAG
«property» es_factible(): bool
bool
Analista
ServicioEscenarios
ServicioOptimizacion
ServicioGenetico
ServicioComparacion
BaseDatos
Session (SQLAlchemy)
fila: FilaComparacion
lote: LoteAG
ModeloTransporte
AlgoritmoGenetico
resultado: ResultadoAG

DIAGRAMA DE SECUENCIA 16 DE 16

HU-11 · Exportar el reporte comparativo en PDF


Analista
ServicioSensibilidad
tabla: Tabla
seccion: Seccion
reporte: Reporte
pdf: _Documento
ServicioComparacion
BaseDatos
Session (SQLAlchemy)
opt
[incluye el análisis de sensibilidad]
analizar(escenario: DatosEscenario): AnalisisSensibilidad | None
AnalisisSensibilidad | None
loop
[por cada sección elegida]
Seccion(titulo: str, parrafos: list[str], tablas: list[Tabla])
Seccion
Tabla(columnas: list[str], filas: list[list[str]], alineacion: list[str] | None)
Tabla
Reporte(titulo: str, subtitulo: str, generado_en: datetime, secciones: list[Seccion])
Reporte
report_exporter.generar_pdf(reporte) arma el PDF con fpdf2
_Documento(reporte: Reporte)
_Documento
add_page(): None
None
loop
[por cada sección, párrafo y fila de tabla]
cell(w: float, h: float, text: str): bool
bool
multi_cell(w: float, h: float, text: str): bool
bool
output(): bytearray
footer(): None
bytearray
opt
[hay solución exacta y lote del AG (al descargar)]
registrar(exacta_id: int, ag_id: int, observaciones: str | None): None
sesion(): Iterator[Session]
Session
scalar(statement: Select[tuple[int]]): int | None
int | None
opt
[la pareja no está registrada]
add(instance: Comparacion): None
None
commit(): None
None
None
Analista
ServicioSensibilidad
tabla: Tabla
seccion: Seccion
reporte: Reporte
pdf: _Documento
ServicioComparacion
BaseDatos
Session (SQLAlchemy)